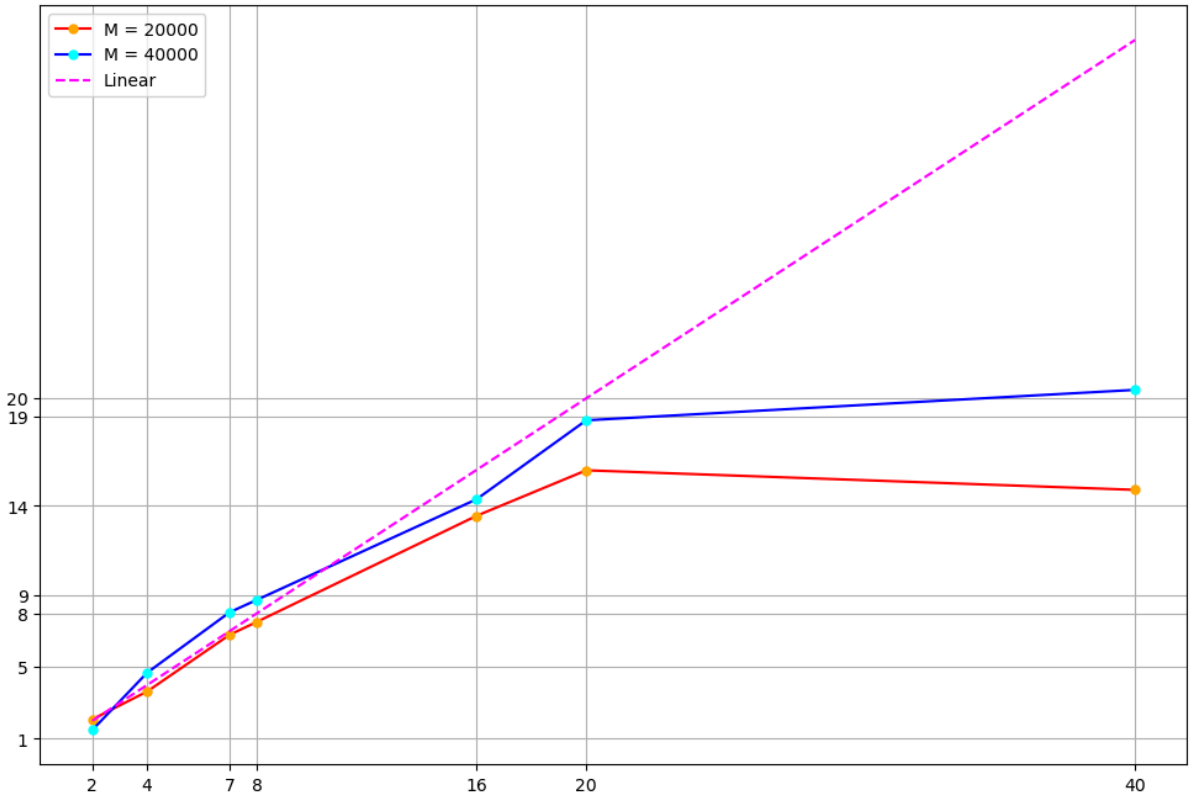
M=N	Количество потоков														
	2			4		7		8		16		20		40	
	T_1	T_2	S_2	T_4	S_4	T_7	S_7	T_8	S_8	T_16	S_16	T_20	S_20	T_40	S_40
20000	3,493237	1,67961	2,0797905	0,960819	3,6356868	0,51399	6,79631315	0,464306	7,5235663	0,259968	13,4371807	0,218604	15,9797487	0,234637	14,88783525
40000	16,358805	10,999388	1,4872468	3,491754	4,6849821	2,031472	8,05268544	1,869353	8,7510518	1,139009	14,3623140	0,871528	18,7702575	0,799195	20,46910329



Отчет по выполнению Задания №1 (Лабораторная работа №3)
Цель работы: Реализация многопоточной версии алгоритма умножения матрицы на вектор (DGEMV) с использованием механизмов стандартной библиотеки C++ (std::async, std::future) и анализ масштабируемости полученного решения.

Описание реализации:
В данной работе была реализована параллельная версия умножения матрицы на вектор. В отличие от использования OpenMP, здесь применена модель асинхронных задач (std::async). Основные особенности реализации:

1. Параллельная инициализация: Матрица и векторы инициализируются в нескольких потоках, что позволяет распределить нагрузку на подсистему памяти с самого начала работы программы.

2. Декомпозиция данных: Матрица распределяется между потоками по строкам (row-wise decomposition). Каждая задача вычисляет свой блок результирующего вектора.

3. Использование std::future: Синхронизация потоков осуществляется через механизм фьючерсов, что гарантирует завершение всех вычислительных узлов перед выводом итогового времени.

Анализ результатов:
Эксперименты проводились для матриц размерностью 20000×20000 и 40000×40000 при количестве потоков от 1 до 40. Полученные данные сведены в таблицу и отображены на графике ускорения.
На малых размерностях (M=20000): Наблюдается стабильный рост ускорения до 16-20 потоков. Однако при переходе к 40 потокам эффективность падает. Это свидетельствует о том, что накладные расходы на создание и управление большим количеством асинхронных задач начинают преобладать над вычислительной выгодой, а задача окончательно упирается в пропускную способность памяти (memory-bound).

На больших размерностях (M=40000): При увеличении объема данных задача демонстрирует лучшую масштабируемость. Пиковое ускорение достигается на 40 потоках (S ≈20.4). Тем не менее, график наглядно показывает существенное отклонение от идеальной линейной зависимости (Linear) начиная уже с 8-16 потоков.
Выводы:

- Реализация на std::async позволяет эффективно утилизировать ресурсы многоядерных систем, однако требует аккуратного подбора количества задач.

- Основным сдерживающим фактором масштабируемости для алгоритма DGEMV остается пропускная способность канала «процессор-память». Поскольку отношение количества арифметических операций к объему передаваемых данных невелико (O(N^2) операций на O(N^2)) данных, вычислительные ядра простаивают в ожидании данных из оперативной памяти.